



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251187
Nama Lengkap	Benedictina Andhika Chinor J. S
Minggu ke / Materi	09 / Pengolahan String dan Regular Expression

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pengantar String

String adalah kumpulan karakter yang membentuk satu kesatuan dan digunakan untuk menyimpan teks dalam program, baik pendek maupun panjang. Data ini disimpan dalam kode ASCII dan sering dianggap sebagai gabungan dari beberapa karakter (array/list karakter), sehingga bukan termasuk tipe data dasar karena menyimpan lebih dari satu nilai dalam satu kesatuan.

Pengaksesan String dan Manipulasi String

String dapat dibuat dengan sederhana menggunakan variabel:

```
namasaya = "Antonius Rachmat C"  
temansaya1 = "Yuan Lukito"  
temansaya2 = 'Laurentius Kuncoro'  
temansaya3 = "Matahari" + 'Bakti'  
  
print(temansaya3)  
print(namasaya[0]) #'A'  
print(namasaya[9]) #'R'  
print(temansaya1[1]) #'u'  
  
huruf = temansaya2[0]  
print(huruf) #'L'
```

Operator dan Metode String

OPERATOR in

Kita dapat memeriksa apakah suatu kalimat merupakan substring dari suatu kalimat lain dengan menggunakan operator in. Hasil dari operator ini adalah True / False.

```
kalimat = "saya mau makan"  
data = "saya"  
print(data in kalimat) #True  
print("mau" in kalimat) #True  
print("dia" in kalimat) #False
```

String juga dapat dilakukan perbandingan (comparison) yang juga menghasilkan True atau False

```
if "saya" > "dia":  
    print("Ya") #Ya  
else:
```

```
print("Tidak")

if "dua" == "dua":
    print("Sama") #Sama
```

FUNGSI len

```
kalimat = "universitas kristen duta wacana yogyakarta"
print(len(kalimat)) #output 42

terakhir = kalimat[len(kalimat)-1]
print(terakhir) #output 'a'

#bisa juga menggunakan indeks -1
terakhir_versi2 = kalimat[-1]
print(terakhir_versi2) #output 'a'
#atau menggunakan indeks -2 untuk huruf terakhir kedua
terakhir2 = kalimat[-2]
print(terakhir2) #output 't'
```

TRAVERSING STRING

Dilakukan dengan akses terhadap indeks:

```
kalimat = "indonesia jaya"
i = 0
while i < len(kalimat):
    print(kalimat[i],end='')
    i += 1
```

Dilakukan tanpa akses terhadap indeks secara otomatis:

```
kalimat = "indonesia jaya"
for kal in kalimat:
    print(kal,end='')
```

STRING SLICE

```
kalimat = "cerita rakyat"
awal = 0
akhir = 6
print(kalimat[awal:akhir]) #cerita
print(kalimat[7:len(kalimat)]) #rakyat
print(kalimat[:5]) #cerit
print(kalimat[5:]) #a rakyat
print(kalimat[:]) #cerita rakyat
```

Operator * dan + pada String

```
kata1 = "saya"
kata2 = "makan"
kata3 = kata1 + " " + kata2
print(kata3) #hasil adalah penggabungan: saya makan
kata4 = "ulang"
print(kata4 * 4) #hasil adalah ulangulangulangulang
kata4 = "ulang "
print(kata4 * 2) #hasil adalah ulang ulang
```

Parsing String

```
kalimat = "Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka"

hasil = kalimat.split(" ")

for kal in hasil:
    if kal[0].isdigit():
        hasil2 = kal.split("-")
print(hasil2[1]+"/"+hasil2[0]+"/"+hasil2[2])
```

Hasil dari keluaran program adalah 08/17/1945

Pengantar Regex

Regular expression (regex) adalah pola berupa kumpulan karakter yang digunakan untuk mencari, mencocokkan, atau memproses pola tertentu dalam sebuah string. Regex dapat digunakan untuk mencari, mengganti, maupun menghapus bagian teks sesuai pola yang diinginkan, sehingga memudahkan proses pengolahan string dibandingkan hanya menggunakan fungsi seperti **split()** atau **find()**.

Dengan menggunakan file mbox-short.txt, kita akan mencoba menampilkan semua string pada file tersebut yang mengandung pola "From:

```
import re
handle=open('mbox-short.txt')
count = 0
for line in handle:
    line=line.rstrip()
    if re.search('From:', line):
        count += 1
        print(line)
print("Count: ",count)
```

Dari kode di atas kita dapat melihat bahwa `re.search` bisa saja diganti dengan menggunakan perintah `find()` pada string biasa

```
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
From: gsilver@umich.edu
From: gsilver@umich.edu
From: zqian@umich.edu
From: gsilver@umich.edu
From: wagnermr@iupui.edu
From: zqian@umich.edu
From: antranig@caret.cam.ac.uk
From: gopal.ramasammycook@gmail.com
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: louis@media.berkeley.edu
From: ray@media.berkeley.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
Jumlah: 27
```

Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Library Python

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{\}	Karakter dengan arti khusus dan escaped character	\{d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
\$	Dakhiri dengan	this\$	Diakhiri dengan kata this
*	0 s/d tak terhingga karakter	\{d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{d+	Minimal 1 s/d tak terhingga karakter
{ }	Tepat sebanyak yang ada para { }	\{d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(sayalkamu)	saya atau kamu sebagai satu kesatuan
	atau	\{d \{ }s	1 digit atau 1 spasi

Tabel 9.1: Special Character pada Python. Sumber: <https://sl1nk.com/48mwd2>

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\b in" "Rain\b"
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\w
\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\AThe"
\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\Z"

Tabel 9.2: Escaped Character pada Regex. Sumber: <https://sl1nk.com/48mwd2>

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b,m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari pola 2 huruf: 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari pola 1 huruf a-Z

Tabel 9.3: Himpunan Karakter pada Regex. Sumber: <https://sl1nk.com/48mwd2>

Nama Fungsi	Kegunaan
findall	mengembalikan semua string yang sesuai pola (matches)
search	mengembalikan string yang sesuai pola (match)
split	memecah string sesuai pola
sub	mengganti string sesuai dengan pola yang cocok

Tabel 9.4: Fungsi Regex pada Python. Sumber: <https://sl1nk.com/48mwd2>

BAGIAN 2: LATIHAN MANDIRI (60%)

Link Github: https://github.com/chinorjodie/71251187_chinor

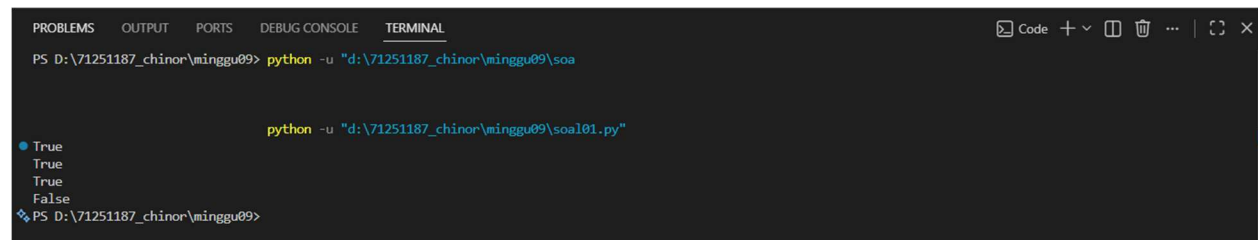
SOAL 1

Source code:

```
import re
def anagram(s1, s2):
    return sorted(re.sub(r'^a-zA-Z', '', s1).lower()) ==
sorted(re.sub(r'^a-zA-Z', '', s2).lower())

print(anagram("mata", "atma"))
print(anagram("mata", "tama"))
print(anagram("mata", "taam"))
print(anagram("mata", "makan"))
```

Output:



```
PROBLEMS OUTPUT PORTS DEBUG CONSOLE TERMINAL
PS D:\71251187_chinor\minggu09> python -u "d:\71251187_chinor\minggu09\soa
python -u "d:\71251187_chinor\minggu09\soal01.py"
True
True
True
False
PS D:\71251187_chinor\minggu09>
```

Penjelasan:

Program itu mendefinisikan fungsi **anagram** untuk mengecek apakah dua string adalah anagram. Caranya, setiap string dibersihkan dulu dengan **re.sub(r'^a-zA-Z', '', s)** supaya hanya huruf yang tersisa, lalu diubah ke huruf kecil dengan **.lower()**. Setelah itu huruf-hurufnya diurutkan dengan **sorted()**. Dua string dianggap anagram kalau hasil urutan hurufnya sama. Jadi ketika memanggil **anagram("mata", "atma")**, huruf yang tersisa setelah diproses adalah **['a','m','t','a']** dan **['a','t','m','a']**, keduanya setelah diurutkan jadi **['a','a','m','t']**, sehingga hasilnya **True**. Begitu juga dengan **"mata"** dan **"tama"**, atau **"mata"** dan **"taam"**. Sedangkan **"mata"** dan **"makan"** tidak sama setelah diurutkan, jadi hasilnya **False**.

SOAL 2

Source code:

```
import re

def hitung_word(s, target):
    huruf = re.findall(r'\b' + re.escape(target) + r'\b', s,
re.IGNORECASE)
    return len(huruf)
sentences = input()
cari = input()

print(f"{cari} ada {hitung_word(sentences, cari)} buah")
```

Output:



```
PS D:\71251187_chinor\minggu09> py .\soal02.py
"Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan"
makan
makan ada 3 buah
PS D:\71251187_chinor\minggu09>
```

Penjelasan:

Program ini mendefinisikan fungsi **hitung_word** yang tujuannya menghitung berapa kali sebuah kata tertentu muncul dalam sebuah kalimat. Caranya, fungsi menggunakan regex **\b** untuk memastikan pencarian dilakukan pada batas kata, sehingga kata yang dicari tidak akan dihitung kalau hanya muncul sebagai bagian dari kata lain. Kemudian **re.escape(target)** dipakai supaya kata target aman dimasukkan ke pola regex meskipun mengandung karakter khusus. Fungsi **re.findall(..., s, re.IGNORECASE)** akan mengembalikan semua kemunculan kata target dalam string **s** tanpa membedakan huruf besar atau kecil, lalu **len(huruf)** menghitung jumlahnya.

Di bagian utama program, pengguna diminta memasukkan sebuah kalimat dan kata yang ingin dicari. Program lalu memanggil **hitung_word** dan mencetak hasil berupa jumlah kemunculan kata tersebut.

SOAL 3

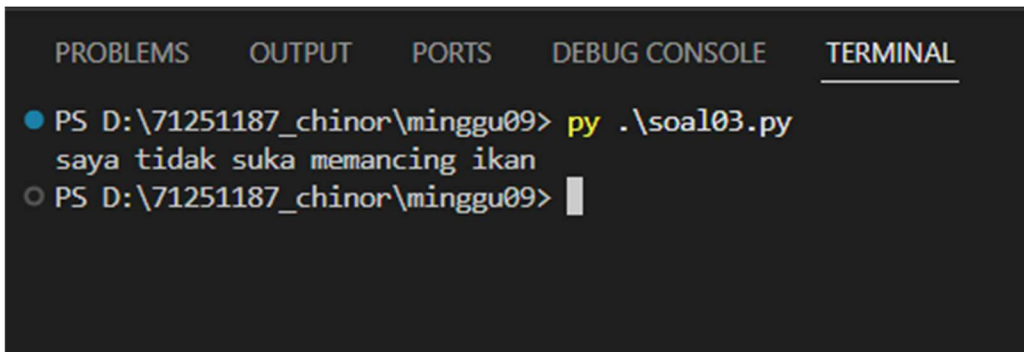
Source code:

```
import re

def hapus_spasi(s):
    s = re.sub(r'\s+', ' ', s).strip()
    return s

print(hapus_spasi("saya tidak suka memancing ikan "))
```

Output:



The screenshot shows a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'PORTS', 'DEBUG CONSOLE', and 'TERMINAL'. The 'TERMINAL' tab is active. The terminal shows the following commands and output:

```
PS D:\71251187_chinor\minggu09> py .\soal03.py
saya tidak suka memancing ikan
PS D:\71251187_chinor\minggu09>
```

Penjelasan:

Program ini digunakan untuk menghapus spasi berlebih dalam sebuah string. Fungsi **hapus_spasi(s)** menerima input berupa teks, lalu menggunakan **re.sub(r'\s+', ' ', s)** untuk mengganti semua spasi yang berjumlah lebih dari satu menjadi satu spasi saja. Setelah itu, fungsi **.strip()** digunakan untuk menghapus spasi di awal dan akhir kalimat. Hasil akhirnya string yang lebih rapi tanpa spasi berlebih, kemudian ditampilkan menggunakan **print()**.

SOAL 4

Source code:

```
import re

def perbandingan(s):
```

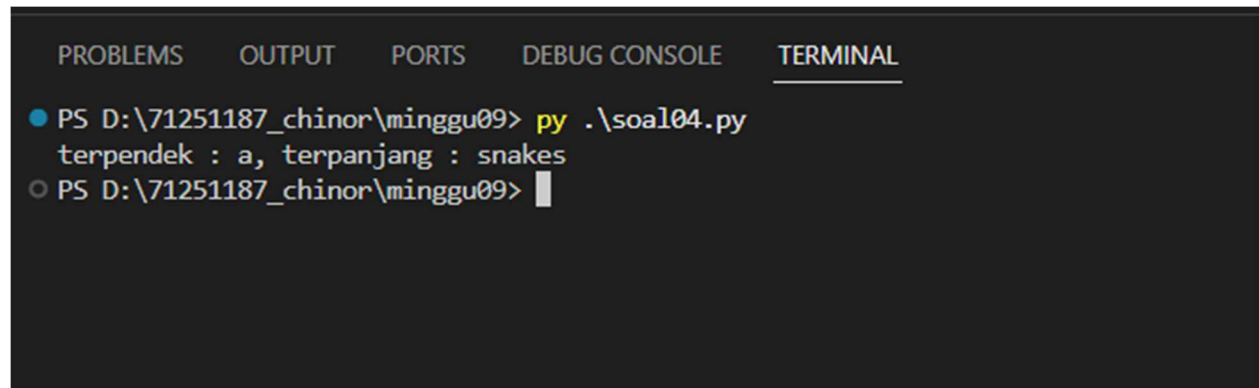
```

huruf = re.findall(r'\b\w+\b', s)
minimum = min(huruf, key=len)
maksimum = max(huruf, key=len)
return f"terpendek : {minimum}, terpanjang : {maksimum}"

print(perbandingan("red snakes and a black frog in the pool"))

```

Output:



```

PROBLEMS OUTPUT PORTS DEBUG CONSOLE TERMINAL
● PS D:\71251187_chinor\minggu09> py .\soal04.py
  terpendek : a, terpanjang : snakes
○ PS D:\71251187_chinor\minggu09>

```

Penjelasan:

Program digunakan untuk mencari kata terpendek dan terpanjang dalam sebuah kalimat. Fungsi **perbandingan(s)** menerima input berupa string, lalu menggunakan **re.findall(r'\b\w+\b', s)** untuk mengambil semua kata dalam kalimat tersebut. Setelah itu, **min()** digunakan untuk mencari kata dengan panjang paling sedikit, sedangkan **max()** digunakan untuk cari kata terpanjang paling banyak, keduanya berdasarkan jumlah karakter (**len**).

SOAL 5

Source code:

```

import re
# hi.strftime('%d-%m-%Y')
from datetime import datetime
def hai(s):
    baca = re.findall(r'\d{4}-\d{2}-\d{2}', s)

```

```

hari_ini = datetime.now()
hasil = []
for i in baca:
    hi = datetime.strptime(i, '%Y-%m-%d')
    selisih = (hari_ini - hi).days
    mantap = hi.strftime('%d-%m-%Y')
    hasil.append(f'{mantap} {hi.time()} selisih {selisih} hari')

return hasil

```

```

text = "Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan nasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Pattimura (TL: 1783-06-08) dan Ki Hajar Dewantara (1889-05-02)."

```

```

for j in hai(text):
    print(j)

```

Output:

```

PROBLEMS OUTPUT PORTS DEBUG CONSOLE TERMINAL
● PS D:\71251187_chinor\minggu09> py .\soal05.py
17-08-1945 00:00:00 selisih 29463 hari
11-11-1785 00:00:00 selisih 87815 hari
08-06-1783 00:00:00 selisih 88702 hari
02-05-1889 00:00:00 selisih 50023 hari
○ PS D:\71251187_chinor\minggu09>

```

Penjelasan:

Program tersebut digunakan untuk mencari tanggal dalam format YYYY-MM-DD di dalam sebuah teks, lalu menghitung selisih hari antara tanggal tersebut dengan tanggal hari ini. Fungsi **hai(s)** menggunakan **re.findall()** untuk mengambil semua tanggal yang sesuai pola, kemudian setiap tanggal diubah menjadi tipe data **datetime** menggunakan **strptime()**. Setelah itu, program menghitung selisih hari antara tanggal sekarang dan tanggal yang ditemukan, lalu mengubah format tanggal menjadi DD-MM-YYYY menggunakan **strftime()**. Hasil akhirnya berupa daftar teks

yang berisi tanggal yang sudah diformat ulang beserta selisih harinya, kemudian ditampilkan satu per satu menggunakan perulangan **for**.

SOAL 6

Source code:

```
import re
import random
import string

def buat_password(length=8):
    karakter = string.ascii_letters + string.digits
    return "".join(random.choice(karakter) for _ in range(length))

def ekstraksi(s):
    emails = re.findall(r'\b[\w\.-]+@[\w\.-]+\.\w+\b', s)
    hasil = []
    for i in emails:
        useruput = i.split('@')[0]
        passwer = buat_password()
        hasil.append(f'{i} username: {useruput} password: {passwer}')
    return hasil

teks = "Berikut adalah daftar email dan nama pengguna dari mailing list: anton@mail.com dimiliki oleh antonius budi@gmail.co.id dimiliki oleh budi anwari slamet@getnada.com dimiliki oleh slamet slumut matahari@tokopedia.com dimiliki oleh toko matahari"

nyam = ekstraksi(teks)

for yyi in nyam:
    print(yyi)
```

Output:

```
PROBLEMS OUTPUT PORTS DEBUG CONSOLE TERMINAL
● PS D:\71251187_chinor\minggu09> py .\soal06.py
anton@mail.com username: anton password: 9vAjT9r0
budi@gmail.co.id username: budi password: 9zIXuWJs
slamet@getnada.com username: slamet password: 9GRtGVhF
matahari@tokopedia.com username: matahari password: P3FYfDm1
○ PS D:\71251187_chinor\minggu09> █
```

Penjelasan:

Program tersebut digunakan untuk mengekstrak alamat email dari sebuah teks, lalu membuat username dan password secara otomatis. Fungsi **ekstraksi(s)** menggunakan **re.findall()** untuk mencari semua email dalam teks sesuai pola tertentu. Setelah itu, setiap email diproses dengan mengambil bagian sebelum tanda **@** sebagai username menggunakan **split()**. Selanjutnya, fungsi **buat_password()** digunakan untuk menghasilkan password acak yang terdiri dari huruf dan angka dengan panjang tertentu (default 8 karakter). Hasilnya kemudian disusun dalam bentuk teks yang berisi email, username, dan password, lalu disimpan dalam list. Terakhir, semua hasil tersebut ditampilkan satu per satu menggunakan perulangan **for**.